

你好，GitHub Actions

你好，GitHub Actions

大家好，我是 Wxh16144，通过学习 Ant Design 的组件库和参与社区贡献，我发现了一些提高开发效率和代码质量的工具。借此机会，希望与大家分享我的经验，帮助大家更好地了解 Ant Design，并将这些技巧应用到自己的项目中。

前言

Ant Design 以开源的形式托管在 GitHub，方便更好的与全球开发者进行交流和合作，也方便开发者提交 issue 和 PR。同时借助 GitHub Actions 和 CI/CD 能力，使得我们更好的管理代码仓库和自动化测试、部署等工作流程，本文将着重介绍 Actions 提供的能力。

什么是 GitHub Actions

GitHub Actions 是一个自动化软件开发工作流程的平台，从想法构建到生成，开发者只需在 `.github/workflows` 目录中添加 `yml` 格式文件，定义 Workflow（工作流程）去实现 CI（持续集成）通过了解 GitHub Actions，我们可以掌握 Workflow 中一些概念。

- **Event(触发事件)**: 触发运行事件，例如，有人创建了 issue、PR 或者推送了代码到某个分支。
- **Job(作业)**: 一个 Workflow 包含一个或多个 **Job**，默认情况下并行运行，我们可以设置让其按顺序执行，每个 **Job** 可以包含多个 **Step**。
- **Step(步骤)**: 定义每一个部分的工作内容，每一个 **Step** 都是一个单独的进程运行。该部分下每个项目都是一个单独操作或者 shell 脚本。

引用官方文档的 Workflow 图，我们可以直观的看懂 **Event**、**Job** 和 **Step** 之间的关系：

图片说明: `overview-actions-simple`

如何使用

通过上述了解，我们可以知道 Ant Design 的所有 Workflow 都放置在 `.github/workflows` 目录中进行管理。

Ant Design 的 CI 覆盖了以下几个方面：

- **社区管理**：使用 GitHub Actions 进行 issue/PR 质量检查，通过评论和标签来提高 issue/PR 的质量，提高协作效率。
- **代码质量**：使用 ESLint 和 Prettier 进行代码规范检查，以确保代码质量和一致性。
- **测试**：使用 Jest 和 testing-library 进行单元测试和快照测试，以确保代码的正确性和稳定性。
- **构建**：构建 ES5 和 ES6 两种模块规范的文件，以确保库能在不同的环境下使用。
- **部署**：使用 dumi 自动生成文档并发布到 GitHub Pages 上。

Issue

issue 作为 GitHub 平台上的一个功能，它像一个信息汇总中心一样，收集社区反馈的问题。允许 Collaborator 添加标签、里程碑、指派人员等信息，以便更好地组织任务和项目。

保证 issue 质量

为了确保 issue 包含足够的信息，帮助 Ant Design 团队对 issue 进行分析和优先级排序，我们提供了 issue 助手 来规范创建 issue 的流程。同时，利用 GitHub Actions 对创建的 issue 进行检查。未通过助手创建的 issue 将会被关闭，并打上 Invalid 标签，然后以评论的形式提醒创建者需要如何进行提问。就像这样：

图片说明：invalid-issue-preview

但即便有时候使用了 issue 助手，团队成员可能也无法从提供的内容中得到有效的信息，这时候会选择手动对 issue 添加 `Need Reproduce`、`needs-more-info` 或 `help wanted` 等标签进一步把控 issue 质量，在 `issue-labeled.yml` 文件中，记录了不同的标签触发对应的评论回复 Job：

图片说明：need-reproduce-auto-comment-preview

图片说明：help-wanted-auto-comment-preview

常见 issue 答疑

对于一些常见的 issue，团队提供了详细的解答，以帮助开发者更快地解决问题。例如 issue 的 title 中包含有 官网、网站、挂了、IE 等类似关键词时，在 `issue-open-check.yml#L43-L94` Job 中详细记录了标准回复格式，并且将自动关闭 issue。

定期清理 issue

使用 GitHub Actions 定时任务来帮助管理和关闭 issue，这些自动化操作可以有效避免过多的未处理 issue 堆积。

- `issue-close-require.yml`：定时检查被标记为 `Need Reproduce` 或 `needs-more-info` 的 issue，如果超过 3 天没有移除这些标签，则会自动评论并关闭 issue。
- `issue-check-inactive.yml`：每隔 15 天定时检查 30 天内没有任何活动的 issue，并将其添加 `Inactive` 标签，但不会关闭 issue。如果被修改或有新评论，则会自动移除 `Inactive` 和 `needs-more-info` 标签。

图片说明：inactive-issue-preview

Pull Request

Ant Design 团队非常鼓励社区参与 Pull Request (PR)，可以先阅读《贡献者开发维护指南》文档，注意 PR 提交时需要遵守一些规范以确保质量和沟通。同时，也会利用 GitHub Action 对 PR 进行一些要求和审核，以保证代码质量和项目的长期维护。

PR 预检

发起一个 PR 时，通过 PR 模板会自动生成描述内容，其中就包括更新日志这栏，需要开发者进行填写。`pr-open-check.yml` 这个 Job 将会对其进行检查，倘若未填，CI 将会以评论的方式进行提醒。就像这样：

图片说明：pr-non-changelog-comment-preview

同时如果 PR 描述中提及的 issue 带有 `Collaborate PR only`（只允许核心成员维护）标签时，也将会关闭 PR 并评论提醒。

`verify-files-modify.yml` 这个 Job 将会检查 PR 修改内容，如果包含特定目录（如：`./github/` 和 `scripts/`）或特定文件（如：`CHANGELOG.md`）则谢绝社区贡献，将自动关闭 PR 且指定给核心成员。

代码规范检查

在 lint Job 中，总是遵循着对每一位开发者提交的代码进行 lint 检查的流程。

图片说明：eslint-ci-preview

PR 部署预览

每创建一个 PR 时，利用 GitHub Action 自动尝试构建和部署该 PR。这样既可以确保文档正常，又可以预览该 PR 是否会对文档或者组件 Demo 产生影响。PR 部署分为多个 Job，具体流程如下：

- 首先触发 preview-start.yml Job 对 PR 进行一个占位评论，告知开发者正在进行预览构建。也就是大家经常看到的 Preview Preparing...

图片说明：preview-preparing..

- 同时 preview-build.yml Job 会对 site 进行构建操作。
- 最后 preview-deploy.yml Job 会等待 preview-build.yml 运行完成后进行对应的操作，如果构建成功则利用 Surge 进行部署，部署地址规则：`https://preview-{PR-id}-ant-design.surge.sh`，并将之前评论中占位图片修改为构建成功样式（点击该图片即可跳转具体地址）反之则标记为构建失败的图片。

其他审查

- size-limit.yml Job 则是对 PR 的一个产物大小进行一个检查。
- 最近比较火热的 chatGPT，团队也将它添加到 GitHub Action 中，用 AI 先对代码进行审查，具体 Job 可以参考 chatgpt-cr.yml 文件。

单元测试

单元测试作为组件库质量保证最重要的一环，当任何提交推送时都将触发该 CI 进行自动化测试，包括每位开发者发起的 PR，或者主分支更新。

构建测试

团队希望每次代码更新后，都能正常构建打包产物，Ant Design 在 test.yml 文件中添加了 Dist Job 和 Compile Job 以保证仓库可以进行正常打包构建。

功能测试

大家可能有留意到每次仅运行测试相关的 Job 就有多达 30 个。

图片说明：test-jobs-preview

团队对于单元测试的态度非常谨慎，需要考虑组件在 React 的各个主要版本上的运行情况（通常为 16、17 和 18 这三个版本）如果是主分支的更新，还需要考虑项目构建产物（通常为 dist、es 以及 lib）在三个 React 版本上的运行情况。目前已知 Ant Design 所有

组件共有 4000 多个测试用例。为了进一步提高测试效率，我们还搭建了分布式测试环境。

所有这些功能都得益于 GitHub Action 的 Job 矩阵策略，使得我们可以一次性配置多个 Job 来执行测试任务，Normal test 和 Module test 是 Ant Design 利用矩阵策略测试相关的 Job。

网站部署

这里的部署构建部分和前面提到的 PR 预览部署构建行为一致，只不过构建后产物部署目标有所差异。

官网部署

<https://ant.design> 官网使用 GitHub 提供的免费 GitHub Pages 功能，利用 Actions Deploy to GitHub Pages Job 直将构建的文档产物推送到 gh-pages 分支实现。

独立版本

大家都知道 <https://ant.design> 官网永远保持最新版本，但有时候仍还是需要查阅具体版本的文档，Deploy to Surge Job 则是每次发布新版本后将站点部署到 Surge，URL 规则为 `https://ant-design-{major}-{minor}-{patch}.surge.sh` 并将 url 评论在每一个发版 commit 上：

图片说明：everyone-version-preview

其他

上面的篇幅已经讲述了 Ant Design 利用 CI/CD 完成的大部分核心内容，但实际上还有一些 Job 没有具体介绍，这里再补充一些。

接入 IM 通知

为保证开发者和社区成员可以第一时间了解到相关信息，利用 Action 提供的 Event 实现 IM 接入：

- issue-notice、discussion-notice Job 表示每当创建了 Issue、Discussion 通知到钉钉社区群中。
- release-helper.yml CI 文件表示每当 antd 发布版本且创建 Release 时，将更新日志发布到钉钉社区群中。
- 其他未提及到的 Job 等待着大家去探索与发现...

接入自己项目

前面向大家介绍了 Ant Design 使用 GitHub Action 的诸多场景，我们不妨实际动手尝试一下，运用到自己项目中，提高生产效率。下面通过一个简单的 Demo 进行演示。

创建项目

通过命令行 `pnpm create vite@latest my-react-app --template react-ts` 创建一个 Vite + React 项目。

配置 CI Workflow

在项目根目录里 `.github/workflows` 文件夹中新建 `ci.yml`，代码如下：

```
name: CI

# en: Set the event to pull request event and push event of the master branch
# zh: Event 设置为 master 分支的 pull request 事件和 push 事件
on:
  push:
    branches: [master]
  pull_request:
    branches: [master]

permissions:
  contents: write

jobs:
  CI:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code (检出代码)
        uses: actions/checkout@v4

      - name: Setup Utool
        uses: utooland/setup-utoo@v1

      - name: Install pnpm (安装 pnpm)
        uses: pnpm/action-setup@v2
        with:
          version: 7.0.0
```

- name: Install dependencies (安装依赖)
run: pnpm install

- name: lint (代码检查)
run: pnpm run lint

en: The template does not contain test cases. If you need to use test cases, you can uncomment it

zh: 初始化模板中不包含测试用例，如需使用测试用例，可取消注释

- name: Test (测试)

run: pnpm run test

- name: Build (构建)
run: pnpm run build

- name: Upload build artifacts (上传构建产物)
uses: actions/upload-artifact@v3
with:
 name: dist
 path: ./dist

- name: Deploy to GitHub Pages (部署到 GitHub Pages)
uses: peaceiris/actions-gh-pages@v3
en: Only deploy when the push event of the master branch is triggered
zh: 仅在触发 master 分支的 push 事件时部署
if: github.ref == 'refs/heads/master'
with:
 github_token: \${{ secrets.GITHUB_TOKEN }}
 publish_dir: ./dist

上述 Workflow 中包含了一个 CI Job，当我们推送到 master 分支时，CI 触发后将依次运行 lint、build 以及 deploy，如下所示：

图片说明：test-CI-preview

添加缓存

为了进一步优化安装依赖速度，我们可以添加 pnpm 缓存，然后发起一个 Pull Request 验证上一个步骤：

```
# ...  
- name: create pnpm-lock.yaml (创建 pnpm-lock.yaml)  
  run: pnpm install --frozen-lockfile --ignore-scripts  
  
- name: Get pnpm store directory (获取 pnpm store 目录)  
  id: pnpm-cache  
  shell: bash  
  run: |  
    echo "STORE_PATH=$(pnpm store path)" >> $GITHUB_OUTPUT  
  
- name: Setup pnpm cache (设置 pnpm 缓存)  
  uses: actions/cache@v4  
  with:  
    path: ${ steps.pnpm-cache.outputs.STORE_PATH }  
    key: ${ runner.os }-pnpm-store-${ hashFiles('**/pnpm-lock.yaml') }  
    restore-keys: |  
      ${ runner.os }-pnpm-store-
```

...

可以发现，发起的 PR 也正确触发了 CI Job，并且我们的 pnpm store 也被缓存了起来，之后每次触发 CI 都会根据 pnpm-lock.yaml 内容判断是否直接读取缓存。

图片说明：pr-CI-preview

图片说明：restore-cache

关于上面的 Setup pnpm cache 步骤中，7 天内未被访问的任何缓存条目将会被删除。可以存储的缓存数没有限制，但存储库中所有缓存的总大小限制为 10 GB，更多内容请阅读缓存依赖项以加快工作流程。

图片说明：cache-pnpm-store

总结

本次文章到这里就结束了，希望可以帮助大家更进一步了解 Ant Design，也欢迎大家前往 讨论区 参与讨论和建设。